DLDCA Inlab: Revision

Week 11

November 6, 2026

Introduction

The end-semester exam is based on the assembly and performance tasks from the inlabs. This document lists all of these tasks. Practise each task as it is given in its inlab.

These topics are not in the syllabus: `xmm` registers, SIMD optimization, and `rdtsc`. All the other assembly and performance topics from the inlabs are in the syllabus.

Two of the tasks in the list used `xmm` registers or SIMD. We have simplified these two tasks, and the simplified versions are Task 1 and Task 2 of this lab. Practise the simplified versions of these two tasks.

Tasks to revise

Inlab 04: assembly basics

- **Task 2, Dot Product**: integer loop over two arrays with `imul` and an accumulator.

Inlab 05: functions and the stack

- **Task 2, Caller and Callee Semantics**: `call`/`ret`, caller-saved and callee-saved registers.
- **Lab Exam, Polynomial Evaluation**: recursive Horner's rule.
  Expect to be able to do it without the `xmm` registers, using only general purpose registers. See Task 1.

Inlab 06: syscalls, text I/O and libc

- **Task 2, Divide and conquer**: `div` on `rdx:rax`, quotient and remainder.
- **Task 3, atoi**: ASCII digits to an integer.
- **Task 4, The adder**: read two numbers, add them, print the sum with `my_itoa` and `write`.
- **Task 5, Same adder with `printf`**: libc calls, 16-byte stack alignment, `al` for variadic calls. See Task 3.

Inlab 07: pipelining and `perf`

- **Task 3, General unrolled loop**: unroll by 4, then 2, then 1, so the routine works for any N. See Task 4.
- **Task 4, `perf stat`**: hardware counters and IPC.

Inlab 08: memory hierarchy

- **Task 1, Cache latency curve**: read latency against array size, and the L1, L2, L3 and DRAM steps.
- **Lab Exam, Poly Want a Speedup**: speed up `poly_fast`.
  Expect to be able to do it with loop reordering only, without SIMD. See Task 2.

Tasks for this lab

This lab has four tasks. Each one is a task from an earlier inlab, and together they cover the main topics of the syllabus:

1. Task 1: recursive polynomial evaluation with integers (the stack, and saving registers across `call`). This is the simplified lab exam of Inlab 05.
2. Task 2: loop reordering for `poly_fast` (cache locality). This is the simplified lab exam of Inlab 08.
3. Task 3: the adder with `printf` (`read`, atoi, libc, stack alignment). This is Task 5 of Inlab 06.
4. Task 4: the general unrolled loop in EduMIPS64 (hazards, unrolling). This is Task 3 of Inlab 07.

Note: each of the four tasks in this lab combines several things from earlier inlabs. If you have doubts about the simpler tasks or concepts, get them cleared first. Then do these tasks for revision. Each task has hints to help you revise.

Slides

The lab folder has the slides of Weeks 4 to 8: `Week4-Assembly.pdf`, `Week5-Assembly.pdf`, `Week6-Assembly.pdf`, `Week7-Performance.pdf` and `Week8-Performance.pdf`. Use them to revise the concepts behind each task.

Necessary tools

These are the same tools as in the earlier inlabs:

- Task 1 uses `nasm` and `ld`.
- Task 2 uses `g++` and `taskset`.
- Task 3 uses `nasm` and `gcc`.
- Task 4 uses EduMIPS64. It is installed on the SL machines as the command `edumips64`. On your own Ubuntu machine, install it with `sudo snap install edumips64`, or use the web version at `https://web.edumips.org`.

Run every task from the `11_inlab_planb/` directory. Use `make task1`, `make task2` and `make task3` to build and run Tasks 1 to 3. Task 4 has its own commands, `make check4` and `make task4`, which are described in the task.

Tasks

Task 1: Recursive Polynomial Evaluation (Integer Version)

This is the lab exam of Inlab 05, changed to use only integers. All values are 64-bit integers in general purpose registers. There are no `xmm` registers.

The problem. `coeffs` is an array of n integers and `x` is an integer. Compute

$$\texttt{coeffs}[0] + \texttt{coeffs}[1] \cdot x + \texttt{coeffs}[2] \cdot x^2 + \cdots + \texttt{coeffs}[n-1] \cdot x^{n-1}$$

with Horner's method, which takes x out as a common factor again and again: $c_0 + x(c_1 + x(c_2 + \dots))$. As a recursive function:

```
poly_eval(coeffs, n, x):
    if n == 1:
        return coeffs[0]
    rest = poly_eval(coeffs + 8, n - 1, x)    # 8 bytes = one element
    return coeffs[0] + x * rest
```

Write `poly_eval(int64_t* coeffs{rdi}, size_t n{rsi}, int64_t x{rdx}) -> rax` in `task1/poly_eval.s`. The `_start` code is given. It calls `poly_eval` and exits with the result.

Why the stack is needed. After the recursive call returns, you still need `coeffs[0]` and `x`. But the call is allowed to change `rax`, `rcx`, `rdx`, `rdi`, `rsi` and the other caller-saved registers. So you push the two values before the call and pop them after it. Every level of the recursion pushes its own two values, so the stack holds one pair per level.

The steps. Your function does this:

1. **Base case.** `cmp rsi, 1`. If it is not equal, jump to the recursive case. Otherwise load `coeffs[0]` with `mov rax, [rdi]` and `ret`.
2. **Save.** Load `coeffs[0]` into `rax` and `push rax`. Then `push rdx` to save `x`.
3. **Recurse.** Move to the next element with `add rdi, 8`, make the count one smaller with `dec rsi`, and `call poly_eval`. `x` is still in `rdx`, so it is passed again. After the call, `rax` holds `rest`.
4. **Restore.** Pop in the reverse order of the pushes: `pop rdx` gives back `x`, then `pop rcx` gives back `coeffs[0]`.
5. **Combine.** `imul rax, rdx` computes `x * rest`, and `add rax, rcx` adds `coeffs[0]`. Then `ret`.

Every `push` must have a matching `pop` before `ret`. If it does not, `ret` reads a wrong return address from the stack and the program crashes.

With `coeffs` $= 1, 2, 3, 4$ and $x = 2$, the result is $1 + 2 \cdot 2 + 3 \cdot 4 + 4 \cdot 8 = 49$:

```
$ make task1
...
Return Code: 49
```

Task 2: Loop Order and Cache Locality

This is the lab exam of Inlab 08, changed to use only loop reordering. There is no SIMD part.

`task2/poly.cpp` computes a polynomial for every element of a 2048×2048 grid of floats, $out[i][j] = c_0 + c_1 x + \cdots + c_7 x^7$ with $x = in[i][j]$. The given `poly_slow` does it like this:

```
for (int j = 0; j < N; j++)          // outer loop: columns
    for (int i = 0; i < N; i++) {    // inner loop: rows
        float x = in[i][j];
        ...                          // Horner's rule on x
        out_ref[i][j] = r;
    }
```

Why it is slow. C++ stores a 2D array row by row. So `in[i][0]`, `in[i][1]`, `in[i][2]`, ... are next to each other in memory, and 16 floats fit in one 64-byte cache line. The inner loop of `poly_slow` changes `i`, so every read jumps one full row (2048 floats = 8 KB) ahead. Each read lands in a different cache line, and almost every read misses the cache.

The fix. Write `poly_fast` with the same body, but swap the two loops: `i` in the outer loop and `j` in the inner loop. Now the inner loop walks along one row. One cache miss brings in 16 floats, and the next 15 reads hit the cache. Write the result to `out[i][j]` (not `out_ref`):

```
for (int i = 0; i < N; i++)          // outer loop: rows
    for (int j = 0; j < N; j++) {    // inner loop: columns
        ...                          // same body, writes out[i][j]
    }
```

Edit `poly_fast` only, between the `YOUR CODE BELOW` and `YOUR CODE ABOVE` lines. The program times both functions and checks that `out` matches `out_ref`. The task passes when the output is correct and the speedup is at least 1.5×:

```
$ make task2
slow:   ... ms
fast:   ... ms
speedup: ...x
wrong elements: 0
target (>= 1.5x): PASS
```

Timings change from run to run, so run it two or three times.

Task 3: Calling `printf` from Assembly

This is Task 5 of Inlab 06. The program reads two numbers, one per line, adds them, and prints `sum = <sum>` with `printf`. `printf` and `exit` come from the C library (libc). They are declared at the top of `adder.s` with `extern`, and the makefile links libc for you.

The calling convention (SystemV ABI). Every function call is an agreement between the caller and the function. When someone calls your function, they assume that:

- the arguments are in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, in this order.

When your function returns, they assume that:

- the result is in `rax`;
- `rbx`, `rbp`, `r12` to `r15` and `rsp` have the same values as before the call. If your function uses one of them, push it at the start and pop it before `ret`.

Similarly, when you call a function (your own, or `printf`), you can assume that:

- `rbx`, `rbp`, `r12` to `r15` still hold your values after it returns;
- every other register, such as `rax`, `rcx`, `rdx`, `rsi`, `rdi` and `r8` to `r11`, may have changed. To keep a value across a call, put it in `r12` (or another register from the first list), or push it on the stack.

libc functions need two more things:

- **Stack alignment.** `rsp` must be a multiple of 16 at every `call`. Write `and rsp, -16` as the first instruction of `_start`. It clears the low 4 bits of `rsp`, so `rsp` becomes a multiple of 16. Each `push` or `pop` moves `rsp` by 8, so do not leave an odd number of pushes on the stack when you call.
- **`al` for `printf`.** `printf` takes a variable number of arguments, and it reads `al` to learn how many of them are in `xmm` registers. We pass none, so set it to zero with `xor eax, eax` just before `call printf`.

What to write. `my_readline(buf{rdi}, len{rsi}) -> rax` is given. It reads one line from stdin into `buf` and returns the number of bytes read. You write two things:

1. `my_atoi(buf{rdi}, len{rsi}) -> rax`, as in Inlab 06, Task 3. Start with `total = 0`. For each byte, compute `d = byte - '0'`. Stop at the first byte where `d > 9` (this catches the newline), or when all `len` bytes are done. Otherwise `total = total * 10 + d`. Return `total`.
2. `_start`, which makes these calls in order:

```
and rsp, -16
rax = my_readline(inbuf, 256)      ; rdi = inbuf, rsi = 256
rax = my_atoi(inbuf, rax)          ; rdi = inbuf, rsi = bytes read
r12 = rax                          ; first number, safe across calls
rax = my_readline(inbuf, 256)
rax = my_atoi(inbuf, rax)          ; second number
rax = rax + r12                    ; the sum
printf(fmt, sum)                   ; rdi = fmt, rsi = sum, al = 0
exit(0)                            ; given: rdi = 0, call exit
```

Use `lea rdi, [inbuf]` and `lea rdi, [fmt]` to pass the addresses. Call the libc `exit` function, not the `exit` syscall: the function first writes out the text that `printf` is still holding, then ends the program.

Run it and type the two numbers, one per line:

```
$ make task3
./task3.out
48
18
sum = 66

Return Code: 0
```

Task 4: Loop Unrolling for Any N

This is Task 3 of Inlab 07. The program runs in EduMIPS64.

You remember unrolling from Inlab 07, Task 2: one pass of the loop does the work of several passes, so the branch and the counter update run fewer times. But a loop unrolled by 4 is correct only when N is a multiple of 4. In this task you unroll by 4, then by 2, and then by 1 (no unrolling). You place the three loops one after the other, so that your code enters each of them in turn. The code then works for every N and is still fast.

The program. `task4/general.s` computes $c[i] = a[i] + b[i]$ for N elements of 8 bytes each. The setup block is given and must not change. After it, the registers hold:

- `r1`, `r2`, `r3`: pointers to the current element of a, b and c;
- `r4`: the number of elements that are left.

`task4/loop.s` is the plain loop from Inlab 07. Its body (two `ld`, one `dadd`, one `sd`) is the work for one element.

The three loops. After loop 4 fewer than 4 elements are left, so loop 2 runs at most once. After loop 2 fewer than 2 are left, so loop 1 runs at most once. Loops 2 and 1 therefore need only the test at the top, not a branch back. For example, $N = 9$ runs loop 4 twice, skips loop 2, and runs loop 1 once.

```
        if r4 < 4: goto two
loop4:  c[0..3] = a[0..3] + b[0..3]     ; offsets 0, 8, 16, 24
        r4 = r4 - 4
        r1, r2, r3 = r1 + 32, r2 + 32, r3 + 32
        if r4 >= 4: goto loop4

two:    if r4 < 2: goto one
        c[0..1] = a[0..1] + b[0..1]     ; offsets 0, 8
        r4 = r4 - 2
        r1, r2, r3 = r1 + 16, r2 + 16, r3 + 16

one:    if r4 < 1: goto done
        c[0] = a[0] + b[0]

done:   syscall 0
```

Writing a test. The branches `beq` and `bne` only compare for equality. To test "`r4` < 4", first set a register with `slti r20, r4, 4` (it becomes 1 if `r4` < 4, otherwise 0). Then `bne r20, r0, two` jumps when the test is true, and `beq r20, r0, loop4` jumps when it is false.

Making it fast. Use the hazard rules from Inlab 07, Task 1:

- In loop 4, do all 8 loads first, then the 4 adds, then the 4 stores. Each value is then ready before it is used, so there is no RAW stall. Use different registers for the 4 elements (for example `r5` to `r16`).
- Put each `slti` several instructions before the branch that reads it, for example right after the counter update. Then the branch does not wait for it.

Testing. `make check4` runs `general.s` on $N = 8, 6, 9, 1024, 2047, 4098$ and on the edge cases $N = 0, 1, 3$. The given file passes only $N = 0$. `make task4` prints the cycle counts of `loop.s` and `general.s`, without and with forwarding. On the web version, `make task4-tests` writes the tests to `task4/tests.txt`. Paste one over the data section of your program, as described under *Test data* in Inlab 07.

```
$ make check4
PASS -- task4/general.s N=8
...
PASS -- task4/general.s N=3
```

A well-scheduled solution takes about this many cycles:

	`loop.s`		`general.s`	
N	no forwarding	forwarding	no forwarding	forwarding
8	142	105	74	66
6	110	81	62	54
9	158	117	81	73
1024	16398	12297	5916	5908
2047	32766	24573	11799	11791
4098	65582	49185	23591	23583